

# REMIO

Plataforma de Serviços Domésticos

## DOCUMENTAÇÃO DO REPOSITÓRIO DE CÓDIGO

Protótipo do site — desenvolvimento e estrutura técnica

Trabalho de Conclusão de Curso  
MBA em Gestão Analítica com BI e Big Data  
Universidade de Fortaleza — UNIFOR

Repositório: [github.com/LuizOtavioEF/remio](https://github.com/LuizOtavioEF/remio)  
Site no ar: [luizotavioef.github.io/remio](https://luizotavioef.github.io/remio)

Fortaleza — CE, 2026

## SUMÁRIO

|                                                      |    |
|------------------------------------------------------|----|
| 1 Visão Geral do Repositório.....                    | 3  |
| 2 Tecnologias Utilizadas.....                        | 4  |
| 3 Estrutura de Arquivos.....                         | 5  |
| 4 Descrição dos Arquivos.....                        | 6  |
| 5 Arquitetura e Funcionamento.....                   | 8  |
| 6 Funcionalidades-Chave e Localização no Código..... | 9  |
| 7 Modelo de Dados Simulado.....                      | 10 |
| 8 Painéis de Business Intelligence (BI).....         | 11 |
| 9 Como Executar o Projeto.....                       | 12 |
| 10 Versionamento e Publicação.....                   | 13 |
| 11 Limitações e Evolução para um Sistema Real.....   | 14 |

## 1 VISÃO GERAL DO REPOSITÓRIO

Este documento descreve a estrutura técnica do repositório de código da plataforma Remio, um protótipo de marketplace que conecta clientes a prestadores de serviços domésticos. O projeto foi desenvolvido como parte do Trabalho de Conclusão de Curso do MBA em Gestão Analítica com BI e Big Data, tendo como base o plano de negócios da empresa Smart Serviços e inspiração visual na plataforma Romi.

O repositório reúne sete páginas navegáveis, três painéis de Business Intelligence (BI) e uma base de dados simulada, totalizando aproximadamente 2.700 linhas de código distribuídas em arquivos HTML, CSS e JavaScript. Todo o protótipo funciona no navegador, sem necessidade de servidor ou instalação, o que facilita a demonstração e a avaliação acadêmica.

O código-fonte está publicado em repositório público no GitHub e o site encontra-se hospedado gratuitamente via GitHub Pages, acessível por qualquer dispositivo com acesso à internet.

## 2 TECNOLOGIAS UTILIZADAS

A escolha das tecnologias priorizou a simplicidade, a legibilidade do código para fins de documentação acadêmica e a ausência de dependências complexas. A tabela a seguir resume a pilha tecnológica adotada.

| Tecnologia                       | Função no projeto                                                                                                              |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| HTML5                            | Estruturação das sete páginas da plataforma.                                                                                   |
| CSS3                             | Identidade visual completa: cores, tipografia, componentes, animações e layout responsivo (arquivo único estilo.css).          |
| JavaScript (puro / Vanilla)      | Toda a lógica da aplicação: navegação, papéis de usuário, filtros de busca, fluxo de orçamento, pedidos e pagamento protegido. |
| Chart.js (via CDN)               | Biblioteca gratuita de gráficos utilizada nos painéis de BI (linhas, barras e rosca).                                          |
| Lucide Icons (via CDN)           | Biblioteca de ícones vetoriais usada em toda a interface.                                                                      |
| Google Fonts — Plus Jakarta Sans | Tipografia moderna da identidade visual.                                                                                       |
| localStorage (navegador)         | Persistência local da sessão do usuário e dos pedidos, simulando o papel de um banco de dados.                                 |
| Git e GitHub                     | Versionamento do código e hospedagem do repositório e do site (GitHub Pages).                                                  |

Optou-se por JavaScript puro, sem frameworks como React ou Vue, de modo que o código possa ser lido e citado diretamente no corpo do TCC, sem etapas de compilação ou ferramentas adicionais.

### 3 ESTRUTURA DE ARQUIVOS

O repositório está organizado de forma plana e intuitiva, separando páginas (raiz), estilos (css/), lógica e dados (js/), documentação (docs/) e capturas de tela (prints/).

```
remio/
├── index.html      Home: busca, categorias, destaques, como funciona
├── busca.html      Busca de profissionais com filtros
├── perfil.html     Perfil do profissional + solicitação de orçamento
├── pedidos.html    Meus pedidos + Pagamento Protegido + avaliação
├── parceiros.html  Empresas parceiras (lojas e fornecedores)
├── dashboard.html  Painel de BI (adapta-se ao papel logado)
├── login.html      Entrada simulada com seleção de papel
├── css/
│   └── estilo.css  Identidade visual completa (631 linhas)
├── js/
│   ├── dados.js    Banco de dados simulado (338 linhas)
│   ├── app.js      Lógica da aplicação (682 linhas)
│   └── dashboard.js Dashboards de BI com Chart.js (307 linhas)
├── docs/           Documentação do TCC (DESIGN, PROMPTS, README)
└── prints/         Capturas de tela das páginas
```

## 4 DESCRIÇÃO DOS ARQUIVOS

### 4.1 Páginas HTML

Cada página HTML é enxuta e delega toda a lógica e a montagem de conteúdo dinâmico aos arquivos JavaScript. Um mesmo cabeçalho e rodapé são injetados por código em todas as páginas, garantindo consistência.

| Arquivo        | Linhas | Descrição                                                                                                   |
|----------------|--------|-------------------------------------------------------------------------------------------------------------|
| index.html     | 120    | Página inicial: busca principal, categorias de serviços, profissionais em destaque e seção "como funciona". |
| busca.html     | 63     | Listagem de profissionais com filtros por categoria, cidade, nota e faixa de preço.                         |
| perfil.html    | 85     | Perfil detalhado do profissional e modal de solicitação de orçamento em quatro etapas.                      |
| pedidos.html   | 93     | Acompanhamento de pedidos, modal de Pagamento Protegido e modal de avaliação.                               |
| parceiros.html | 44     | Catálogo de empresas parceiras com busca e filtro por segmento.                                             |
| dashboard.html | 25     | Contêiner do painel de BI, preenchido conforme o papel do usuário.                                          |
| login.html     | 60     | Tela de entrada com seleção de papel (cliente, prestador ou empresa parceira).                              |

### 4.2 Folha de Estilo (css/estilo.css)

Concentra toda a identidade visual em um único arquivo, organizado por seções comentadas. Define as variáveis de cor da marca (laranja como cor primária, grafite para textos e âmbar nos destaques), os componentes reutilizáveis (cabeçalho, cartões, botões, modais, painéis de BI) e as regras de responsividade para telas menores.

### 4.3 Lógica e Dados (js/)

| Arquivo               | Responsabilidade                                                                                                                                                                                                                                                                             |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>dados.js</code> | Base de dados simulada do protótipo. Contém as categorias de serviço, os profissionais cadastrados (com nota, faixa de preço, galeria e avaliações), as empresas parceiras, os pedidos iniciais e os indicadores de BI. Em um sistema real, estes dados viriam de um banco de dados via API. |
| <code>app.js</code>   | Núcleo da aplicação. Monta o cabeçalho e o rodapé, gerencia o papel do usuário (via localStorage), executa a busca e os filtros, controla o fluxo de orçamento em etapas, o ciclo de vida dos pedidos e o Pagamento Protegido.                                                               |

| Arquivo                   | Responsabilidade                                                                                                                                                                   |
|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>dashboard.js</code> | Monta os três painéis de BI conforme o papel logado, renderizando indicadores (KPIs) e gráficos com a biblioteca Chart.js, além do mapa de calor de horários de pico do prestador. |

## 5 ARQUITETURA E FUNCIONAMENTO

A aplicação segue um padrão simples e coeso: as páginas HTML declaram a estrutura e marcam-se com um atributo data-pagina; ao carregar, o arquivo app.js identifica a página atual e executa a função de inicialização correspondente, que busca os dados em dados.js e injeta o conteúdo dinâmico na tela.

### Identificação da página e inicialização (trecho de app.js):

```
const pagina = document.body.dataset.pagina;
if (pagina === 'home')      initHome();
if (pagina === 'busca')     initBusca();
if (pagina === 'perfil')    initPerfil();
if (pagina === 'pedidos')   initPedidos();
if (pagina === 'parceiros') initParceiros();
if (pagina === 'login')     initLogin();
```

Os papéis de usuário (cliente, prestador e empresa parceira) são guardados no localStorage do navegador. Isso permite que o mesmo conjunto de páginas se comporte de maneira diferente conforme quem está logado — em especial o painel de BI, que exibe indicadores distintos para cada perfil.



## 6 FUNCIONALIDADES-CHAVE E LOCALIZAÇÃO NO CÓDIGO

A tabela a seguir relaciona cada funcionalidade central do protótipo ao ponto do código onde está implementada, facilitando a consulta e a citação no TCC.

| Funcionalidade                                        | Onde está implementada                                                                                                                 |
|-------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| Faixa de preço (\$, \$\$, \$\$\$) — estilo iFood      | Campo faixaPreco em dados.js; função precoHTML() e filtro em initBusca(), em app.js.                                                   |
| Pagamento Protegido (custódia / escrow) — estilo Uber | Função initPedidos() em app.js: o valor fica retido e só é liberado na avaliação; a comissão de 15% é calculada no modal de pagamento. |
| Ciclo de vida do pedido                               | Máquina de estados em app.js: aguardando → orçamento → agendado → concluído → avaliado.                                                |
| Papéis de usuário                                     | localStorage gerenciado em app.js; o painel adapta-se em dashboard.js.                                                                 |
| Indicadores de BI                                     | dashboard.js: KPIs animados, gráficos de linha, barra e rosca, e mapa de calor.                                                        |
| Sistema de avaliações                                 | Estrelas clicáveis em pedidos.html; avaliações exibidas em perfil.html.                                                                |
| Solicitação de orçamento em etapas                    | Função initPerfil() em app.js: formulário de quatro etapas com resumo final.                                                           |

## 7 MODELO DE DADOS SIMULADO

O arquivo dados.js cumpre o papel que, em produção, caberia a um banco de dados. Os dados estão organizados em coleções, descritas a seguir.

- **CATEGORIAS** — os oito tipos de serviço doméstico (faxina, elétrica, hidráulica, pintura, pedreiro, jardinagem, ar-condicionado e montagem de móveis).
- **PROFISSIONAIS** — doze prestadores com nome, foto, categorias, cidade, nota, número de avaliações, anos de experiência, faixa de preço, preço médio, selo de verificado, biografia, qualificações, galeria de trabalhos e avaliações recebidas.
- **PARCEIROS** — seis lojas e fornecedores com segmentos, cidade, nota e selo de verificado.
- **PEDIDOS\_INICIAIS** — pedidos de exemplo já em diferentes status, para demonstrar o ciclo completo.
- **BI** — os indicadores que alimentam os três painéis (cliente, prestador e empresa parceira).

### Exemplo da estrutura de um profissional (trecho de dados.js):

```
{
  id: 1, nome: 'Maria das Graças Silva',
  categorias: ['faxina'], cidade: 'Fortaleza, CE',
  nota: 4.9, numAvaliacoes: 132, anosExp: 8,
  faixaPreco: 1, precoMedio: 140,
  verificado: true, pro: true,
  bio: 'Especialista em limpeza residencial...',
  qualificacoes: [ ... ], galeria: [ ... ],
  avaliacoes: [ ... ]
}
```

## 8 PAINÉIS DE BUSINESS INTELLIGENCE (BI)

Os painéis de BI constituem o diferencial do protótipo em relação ao tema do MBA. Cada papel de usuário acessa um conjunto próprio de indicadores, construídos sobre os dados simulados.

| Painel           | Indicadores apresentados                                                                                                                                                                              |
|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Cliente          | Total investido em serviços, número de contratações, nota média atribuída, economia estimada, gastos por mês e contratações por categoria.                                                            |
| Prestador        | Faturamento mensal, serviços concluídos, nota média, taxa de conversão de orçamentos, demanda por tipo de serviço, comparativo de preço com a média da categoria e mapa de calor de horários de pico. |
| Empresa Parceira | Indicações recebidas, contatos gerados, taxa de conversão, avaliação da loja, contatos por mês, interesse por segmento e funil de indicação até venda.                                                |

No protótipo, esses números são estáticos. Em um sistema real, seriam calculados por um pipeline de Big Data, que captura eventos da plataforma, os consolida em um data warehouse e os disponibiliza a uma camada de BI em tempo real.

## 9 COMO EXECUTAR O PROJETO

O protótipo não exige instalação. Há duas formas de acesso:

- **Acesso on-line:** [luizotavioef.github.io/remio](https://luizotavioef.github.io/remio) — basta abrir o endereço no navegador.
- **Acesso local:** baixar o repositório e abrir o arquivo index.html com um duplo clique. É necessária conexão com a internet apenas para carregar as fontes, os ícones e a biblioteca de gráficos.

**Atalhos para demonstração:** é possível entrar diretamente em um painel adicionando o papel à URL, por exemplo `dashboard.html?papel=prestador`, sem passar pela tela de login.

## 10 VERSIONAMENTO E PUBLICAÇÃO

O código é versionado com Git e hospedado no GitHub. A publicação do site é feita automaticamente pelo GitHub Pages a partir da branch principal do repositório. Qualquer alteração enviada ao repositório (commit e push) é refletida no site no ar.

| Recurso               | Endereço                                                                          |
|-----------------------|-----------------------------------------------------------------------------------|
| Repositório de código | <a href="https://github.com/LuizOtavioEF/remio">github.com/LuizOtavioEF/remio</a> |
| Site publicado        | <a href="https://luizotavioef.github.io/remio">luizotavioef.github.io/remio</a>   |

## 11 LIMITAÇÕES E EVOLUÇÃO PARA UM SISTEMA REAL

Por se tratar de um protótipo acadêmico de front-end, o projeto possui limitações conhecidas. A relação a seguir indica o que seria necessário para transformá-lo em um produto de produção — material que também subsidia as seções de arquitetura e de limitações do TCC.

- Banco de dados real (por exemplo, PostgreSQL ou Firebase) substituindo o arquivo dados.js.
- Back-end e API para processar as regras de negócio de forma segura no servidor.
- Autenticação real, com verificação de identidade e de antecedentes dos profissionais.
- Gateway de pagamento (por exemplo, Mercado Pago ou Stripe) para operar a custódia de valores de fato.
- Conformidade com a Lei Geral de Proteção de Dados (LGPD) no tratamento de dados pessoais.
- Pipeline de Big Data (eventos → data warehouse → camada de BI) alimentando os painéis em tempo real.